TROUBLESHOOTING

Troubleshooting

Symptom -> diagnosis -> fix for `scripts/setup-agents-wizard.sh` and the Lumen semantic search stack it configures.

Background reading: README.md (what gets installed), FAQ.md (why it is built this way) and SAFETY-AND-ROLLBACK.md (how to undo a run).

If the embedding backend runs on an Intel iGPU through Vulkan, read OLLAMA-REMEDIATION.md before anything else here — that hardware returns silently-corrupt vectors under HTTP 200, so several symptoms below have one shared cause that no amount of restarting will clear.

Start with these two, whatever your symptom. Both are read-only and safe to run at any time, including during an index build:

```
./scripts/ollama-vulkan-remediation.sh --check     # is the
        backend the corrupting one?
./scripts/lumen-index-doctor.sh "$PWD"             # is the
        existing index already poisoned?
```

The second one matters even when everything looks fine: the worst failure mode has **no symptom at all** — see #14. Reference for both: OPERATIONAL-SCRIPTS.md.

60-second health check

Run this first — it reproduces what the wizard's own `verify_lumen` checks, plus the state of the embedding backend:

```
command -v lumen                                    # -> /home/
        <you>/.local/bin/lumen
lumen version                                       # -> e.g.
        0.0.41
bash -lc 'command -v lumen'                         # must ALSO
        resolve in a login shell
curl -sf http://localhost:11434/api/tags >/dev/null && echo
        backend-ok || echo backend-DOWN
curl -s http://localhost:11434/api/embed \
  -d '{"model":"ordis/jina-embeddings-v2-base-
        code","input":"health check"}' \
  | grep -q '"embeddings"' && echo embed-ok || echo embed-
        BROKEN   # reachable != healthy
ollama list | grep ordis/jina-embeddings-v2-base-code
claude mcp get lumen                                # Claude Code
        registration
jq -r '.mcpServers.lumen | .command + " " + .args[0]' ~/.kimi-
        code/mcp.json ~/.qwen/settings.json
jq -c '.mcp.lumen' ~/.config/opencode/opencode.json
du -sh ~/.local/share/lumen
./scripts/rollback-agents-wizard.sh --list          # the run is
        recorded and undoable
jq -c '.mcpServers.lumen' ~/.claude.json            # the mirror
        MiMo and friends inherit
```

The `jq` lines should print an absolute path ending in `/.local/bin/lumen stdio`; the `du` line, a size for the index store.

Then the two checks the list above cannot make. Every command in it is a *per-item* probe, and the failure that cost this project the most was invisible to exactly that kind of probe:

```
./scripts/ollama-vulkan-remediation.sh --check      # expect
        library=cpu + "batch probe: OK 32/32"
./scripts/lumen-index-doctor.sh "$PWD"              # exit 0 = the
        stored vectors are trustworthy
```

`--check` is read-only and defaults to that, so running it bare is safe. The doctor is read-only too and safe during an index run. See #14.

Quick triage

Symptom	Diagnosis	Fix
`lumen: could not locate the Lumen plugin binary`, exit 127	Wrapper installed, plugin binary absent	Install the Lumen plugin (`/plugin` in Claude Code) or set `LUMEN_BIN` — #1

Symptom	Diagnosis	Fix
`Error: unknown command "serve" for "lumen"`	Stale MCP config still passing `serve`	Re-run the wizard, or change the agent's `args` to `["stdio"]` — #2
Indexing looks stuck; searches keep saying `Index is being updated in the background`	Embedding throughput is the bottleneck; concurrent indexers share one ollama	Watch `pgrep -af "lumen.*index"` and ollama CPU; index one project at a time — #3
`No results found.`	Index still building, or the score threshold cut everything	Wait for indexing; then `--min-score -1 -n 20` — #4
`Embedding backend unreachable` / index and search both fail	ollama not running or not listening where Lumen looks	`systemctl is-active ollama`; `sudo systemctl enable --now ollama` — #5
`Embedding backend is WEDGED …` `returns NaN for every input`, or an HTTP 500 `unsupported value: NaN` followed by Lumen's usage text	The ollama runner is up but broken — `/api/tags` still answers `200`	`ollama stop <model>` for a fresh runner — #5b
The wizard warns `Embedding model is GPU-offloaded and ollama reports library=Vulkan`	The corrupting path. A large **batch** comes back as all-zero — or, worse, as a well-formed *stale duplicate* — under HTTP 200	`./scripts/ollama-vulkan-remediation.sh --check`, then `--apply` — #14 · OLLAMA-REMEDIATION.md
Searches "work" but return plausible nonsense, or miss files you know are indexed. **No error anywhere.**	Stale-duplicate vectors written under HTTP 200. Invisible to NaN / all-zero / L2-norm checks	`./scripts/lumen-index-doctor.sh "$PWD"` — #14
`REFUSING TO START: ollama reports library=Vulkan`	Working as designed — it will	

Symptom	Diagnosis	Fix
(exit 3) from `lumen-reindex.sh`	not write more corrupt vectors	Fix the backend, or pass `--allow-gpu` if you accept the risk — #14
`no Lumen index found for … under …` (exit 2) from `lumen-index-doctor.sh`	No index exists for that project, or `LUMEN_STORE` points elsewhere	Index it first; check `ls ~/.local/share/lumen/*/index.db`
`command -v ashlr` finds nothing	**Expected.** ashlr is a Claude Code plugin with no binary — by design	Check `~/.claude/plugins/cache/ashlr-marketplace/ashlr/`, then run the three slash commands — ACTION-REQUIRED.md
`mimo mcp list` does not show `lumen`	`~/.claude.json` lacks `.mcpServers.lumen` (MiMo inherits from it), or MiMo started before it was added	Re-run the wizard, then restart MiMo. Check with `jq -c '.mcpServers.lumen' ~/.claude.json`
The wizard seems to hang at the very end	It is waiting for **Enter** on the `ACTION REQUIRED` list	Press Enter. For automation, `WIZARD_NONINTERACTIVE=1`
Backend is up but indexing still fails	Embedding model not pulled	`ollama pull ordis/jina-embeddings-v2-base-code` — #6
A file you just created/edited never appears in results	It was created after the indexer's tree walk started	Re-run `lumen index <path>` (incremental) — #7
`jq: command not found`, or the wizard exits at Step 1	`jq` missing and no supported package manager	The wizard auto-installs it; otherwise install manually — #8
Worry about `PATH` growing after repeated runs	Not possible — the block is `case`-guarded	Verify with `tr ':' '\n' <<<"$PATH" | sort | uniq -d` — #9
`lumen: command not found` right after the wizard finished	Current shell never re-read its rc files	`exec bash -l` — #10
Need to file a bug	—	

Symptom	Diagnosis	Fix
		Run the test suite and attach `.test-evidence/<timestamp>/` — #11
The wizard broke something and you want it back	Every run is recorded in a manifest	`./scripts/rollback-agents-wizard.sh --dry-run`, then `--yes` — #12
`Backup missing for <file>` / `No backup sessions found at …`	The session's `files/` copy or the whole state directory is gone, or `WIZARD_STATE_DIR` differs	Check `--list`, match `WIZARD_STATE_DIR`, else uninstall by hand — #13

1. `lumen: could not locate the Lumen plugin binary` (exit 127)

Symptom

```
lumen: could not locate the Lumen plugin binary.

Looked under:
  ~/.claude-shared/plugins/cache/*/lumen/*/bin/
  ~/.claude*/plugins/cache/*/lumen/*/bin/

Is the Lumen plugin installed? In Claude Code run:  /plugin
Or point at the binary directly:  export LUMEN_BIN=/path/to/lumen-linux-amd64
```

Exit status is `127`, deliberately — never `0`. A silent success here would leave agents thinking Lumen works (tests **B7**, **B8**).

Diagnosis

`~/.local/bin/lumen` is only a launcher. The real binary ships with the Claude Code Lumen plugin at a version-pinned path, and the wrapper found nothing to `exec`:

```
ls -d ~/.claude-shared/plugins/cache/*/lumen/*/bin/ 2>/dev/null
ls -d ~/.claude*/plugins/cache/*/lumen/*/bin/       2>/dev/null
uname -s; uname -m       # must map to linux|darwin and amd64|arm64
```

If those globs print nothing, the plugin is not installed. If they print a directory but the error persists, the binary name does not match `lumen-<os>-<arch>` for this host, or it is not executable.

Fix

```
# A. Install the plugin from inside Claude Code
/plugin

# B. Or point the wrapper at a binary directly
export LUMEN_BIN=/path/to/lumen-linux-amd64
lumen version
```

`LUMEN_BIN` always wins over path resolution. If it is set but not executable you get a distinct message and the same exit `127`:

```
lumen: LUMEN_BIN is set but not executable: /nope/missing
```

Two other `127` variants come from the host mapping — `lumen: unsupported OS: ...` and `lumen: unsupported arch: ...`. Those mean the wrapper cannot name a binary for this platform; use `LUMEN_BIN`.

---

2. `Error: unknown command "serve" for "lumen"`

Symptom

The MCP server fails to start on every agent session; agent logs show:

```
Error: unknown command "serve" for "lumen"
Run 'lumen --help' for usage.
```

Diagnosis

Lumen's MCP subcommand is `stdio`. `serve` never existed. An old wizard revision wrote `{"command": "lumen", "args": ["serve"]}` into every agent config, and that config is still on disk — replacing the wizard does not rewrite a config you have since edited by hand.

Check each agent at its **real** config location:

```
claude mcp get lumen                                          #
        Claude Code
for f in ~/.kimi-code/mcp.json ~/.qwen/settings.json; do
  [ -f "$f" ] && printf '%s -> %s\n' "$f" "$(jq -c
        '.mcpServers.lumen' "$f")"
```

```
done
jq -c '.mcp.lumen' ~/.config/opencode/opencode.json           #
        Opencode uses .mcp
```

Correct output has `stdio` and an **absolute** command path:

```
{"command":"/home/<you>/.local/bin/lumen","args":["stdio"]}
{"type":"local","command":["/home/<you>/.local/bin/
        lumen","stdio"],"enabled":true}
```

Fix

Re-run the wizard (it merges with `jq`, backs up first and records the change), or patch in place:

```
for f in ~/.kimi-code/mcp.json ~/.qwen/settings.json; do
  [ -f "$f" ] || continue
  cp -p "$f" "$f.bak.$(date +%Y%m%d%H%M%S)"
  tmp=$(mktemp)
  jq --arg bin "$HOME/.local/bin/lumen" \
     '.mcpServers.lumen = {"command": $bin, "args": ["stdio"]}'
        "$f" > "$tmp" && mv "$tmp" "$f"
done

claude mcp remove lumen -s user
claude mcp add-json lumen "{\"command\":\"$HOME/.local/bin/
        lumen\",\"args\":[\"stdio\"]}" -s user
```

Restart the agent afterwards — MCP servers are launched at session start.

> `codegraph` legitimately uses `serve`. Only the `lumen` entry changes.
>
> **Look in the right file.** A previous revision wrote `~/.claude/mcp.json`, `~/.kimi/config.json`, `~/.opencode/config.json`, `~/.mimo/config.json` and `~/.qwen-code/config.json`. No agent reads any of those — they are orphans, and editing them changes nothing. Delete them and use the paths above.

---

3. Indexing appears stuck, or every search says the index is updating

Symptom

```
Index is being updated in the background. Results may be
incomplete or outdated.
Use grep/glob/find for code search until indexing finishes
```

```
(usually a few minutes;
longer for large repositories).
```

…and it keeps saying that for far longer than “a few minutes”.

Diagnosis

Indexing is not I/O bound on your repository — it is bound on **embedding**. Every chunk is sent to ollama and embedded; on CPU-only hosts that is the whole cost. Two things make it worse:

1. **Concurrent indexers.** Several agents (or several projects) each start their own `lumen index`, and all of them queue against the *same single* ollama backend. They do not fail; they just take turns, so each one appears frozen.
2. **Repository size.** A monorepo can produce a multi-hundred-megabyte `index.db`.

Confirm which one you have:

```
pgrep -af "lumen.*index"                              # how many
        indexers are running, and for what
ollama ps                                             # is a model
        actually loaded and busy
top -b -n1 | head -15
        # ollama pegging cores == it is working
du -sh ~/.local/share/lumen/* | sort -h | tail -5 # the growing
        directory is your project
```

Re-run `du -sh` a minute apart: if the largest directory grew, indexing is progressing and you simply need to wait.

Fix

- Let it finish. Use `grep`/`glob` for code search meanwhile, exactly as the message says.
- Serialize the work: index the big repository once, from the terminal, before opening several agents on it.

  ```
  lumen index /path/to/big/repo
  ```

- For very large repositories, raise the reindex timeout for that shell:

  ```
  export LUMEN_REINDEX_TIMEOUT=1800   # seconds
  ```

- Get per-phase timings to see where the time goes:

  ```
  lumen search "some query" -p /path/to/repo --trace
  ```

- If a previous run was killed mid-flight and nothing is making progress (`pgrep` empty, `du` flat), rebuild that one project:

  ```
  ./scripts/lumen-reindex.sh /path/to/repo --force
  ```

  The wrapper is preferable to a bare `lumen index -f` for a long unattended run: it retries around transient backend faults instead of dying halfway and printing Lumen's usage text, which reads like *you* mistyped the command. Exit codes and flags: OPERATIONAL-SCRIPTS.md.

---

4. Search returns `No results found.`

Symptom

```
$ lumen search "how are user passwords checked" -p /path/to/repo
No results found.
```

Diagnosis

Four plausible causes, in order of likelihood:

1. The index is not finished (see #3) — an empty or partial index has nothing above threshold.
2. The default result count and score threshold are too strict for a vague query. `-n` defaults to `8`, and low-similarity matches are filtered out.
3. You are searching the wrong root. `-p` is the directory to search; `--cwd` is the project root when `-p` points at a subdirectory. Get these wrong and you query a different index.
4. **The vector for that file is corrupt.** If it was embedded by a GPU/Vulkan backend it may be a well-formed but *stale* vector that has nothing to do with the file's content, so no query can ever retrieve it. Rule this out with `./scripts/lumen-index-doctor.sh "$PWD"` — #14.

Fix

Widen the search before concluding anything is broken:

```
lumen search "how are user passwords checked" -p /path/to/repo --
        min-score -1 -n 20
```

`--min-score` accepts `-1` to `1`; `-1` disables the cutoff so you see the ranked tail, which tells you whether the index has content at all. Then tighten back up.

Other useful shapes:

```
lumen search "..." -p /path/to/repo --summary
        # locations only, no snippets
lumen search "..." -p /path/to/repo/sub --cwd /path/to/repo
lumen search "..." -p /path/to/repo -f                  # force a
        full re-index first
```

If `--min-score -1 -n 20` still returns nothing, the project has no index — run `lumen index /path/to/repo` and check #5 and #6.

---

5. Embedding backend unreachable

Symptom

The wizard prints one of:

```
    Embedding backend unreachable at http://localhost:11434 -
Lumen cannot index.
    Embedding round-trip failed at http://localhost:11434 - Lumen
will fail to index.
```

or indexing/searching fails immediately, before any progress.

> If instead you see `Embedding backend is WEDGED …`, the daemon is up but broken — jump to #5b.

Diagnosis

Lumen has no built-in embedder. Without ollama (or LM Studio) reachable, it can neither chunk nor search.

```
curl -sf http://localhost:11434/api/tags >/dev/null && echo
        reachable || echo unreachable
systemctl is-active ollama          # -> active | inactive |
        failed
command -v ollama
echo "${OLLAMA_HOST:-http://localhost:11434}"
```

The test suite's `F3` probes `${OLLAMA_HOST:-http://localhost:11434}/api/tags` with a 5-second timeout, so match your check to that URL. The wizard's own `verify_lumen` starts there too, then goes further: it sends a real embedding request to `/api/embed` (90-second timeout) and requires a vector back. **Reachable is not healthy** — see #5b.

Fix

```
sudo systemctl enable --now ollama      # start now and at boot
systemctl status ollama --no-pager      # if it refuses to start
```

Without systemd, or without root, run it yourself:

```
ollama serve &
```

Pointing at a backend elsewhere (another host, a non-default port):

```
export OLLAMA_HOST=http://<host>:11434
```

Remember the tuning knobs are commented out in the wizard's `~/.bashrc` block by design — if you set `OLLAMA_HOST` for your shell, agent-spawned MCP servers only inherit it if they are started from a shell that has it. See FAQ: why the wizard does not pin these.

5b. Embedding backend is WEDGED (`NaN` for every input)

Symptom

```
   Embedding backend is WEDGED at http://localhost:11434 - it
returns NaN for every input.
    Fix: ollama stop ordis/jina-embeddings-v2-base-code   (a fresh
runner clears it)
```

Or, when you index by hand, an HTTP 500 followed by Lumen's usage text — which reads like you mistyped the command:

```
{"error":"failed to encode response: json: unsupported value:
NaN"}
```

Diagnosis

This is the failure mode that a reachability check cannot see. The daemon is up, `/api/tags` answers `200`, `systemctl is-active ollama` says `active` — and yet **every** embedding request comes back `NaN`. It happens to the ollama *runner* process under sustained load, and it does not clear itself.

Confirm it with a round-trip, exactly as the wizard does:

```
curl -s http://localhost:11434/api/embed \
  -d '{"model":"ordis/jina-embeddings-v2-base-
        code","input":"health check"}'
# healthy -> {"model":"...","embeddings":[[0.01,...]]}
# wedged  -> {"error":"failed to encode response: json:
        unsupported value: NaN"}
```

Fix

Stop the model so ollama starts a fresh runner for it:

```
ollama stop ordis/jina-embeddings-v2-base-code
curl -s http://localhost:11434/api/embed \
  -d '{"model":"ordis/jina-embeddings-v2-base-
        code","input":"health check"}' | head -c 80
```

If that is not enough, restart the daemon (`sudo systemctl restart ollama`). Then re-index — whatever was indexed while the backend was wedged is missing, partial, **or silently wrong**:

```
./scripts/lumen-reindex.sh /path/to/repo --force
./scripts/lumen-index-doctor.sh /path/to/repo      # then confirm
```

> **`ollama stop` is first aid, not a fix, and the `NaN` is the tombstone rather than the fault.** On an Intel iGPU driven by Vulkan, a large embedding **batch** times out the i915 fence; ollama reads the abandoned buffer and returns garbage under **HTTP 200** long before it ever returns `NaN` — sometimes an all-zero vector, and sometimes a **well-formed, unit-norm stale duplicate** that no per-vector check can see. A fresh runner clears the tombstone and the next large batch recreates the whole situation.
>
> One command replaces the manual confirmation below: `./scripts/ollama-vulkan-remediation.sh --check` (read-only, and the default action). By hand: `ollama ps` (`100% GPU`) and `journalctl -u ollama | grep 'inference compute'` (`library=Vulkan`). Then apply the fix — `--apply`, or the tiers in **OLLAMA-REMEDIATION.md**.

> **Then check the index**, because none of that repairs what was already written: #14. Evidence for the loud modes is in OLLAMA-NAN-WEDGE.md; the settled verdict on what reached the index is INDEX-CORRUPTION-RECONCILIATION.md.

Tests **A33** and **A34** keep this check in the wizard: the health probe must use `/api/embed`, and the `NaN` case must be detected and named rather than reported as a generic failure. Tests **A35**–**A41** keep the backend-placement warning honest: GPU residency and the compute library must both be *probed* (`/api/ps`, ollama's `inference compute` line), an unreadable library must be reported as unknown rather than assumed clean, and the wizard must never apply the remediation itself — no service restart, no write to `/etc/sysconfig/ollama`.

---

6. Embedding model missing

Symptom

The backend answers `/api/tags`, but indexing fails or the wizard summary shows:

```
embedding model (ordis/jina-embeddings-v2-base-code)
```

Diagnosis

```
ollama list | grep ordis/jina-embeddings-v2-base-code
```

No row means the model was never pulled — most often because the pull failed while the wizard ran (it only warns: `Model pull failed - Lumen indexing stays broken until it succeeds.`).

Fix

```
ollama pull ordis/jina-embeddings-v2-base-code
```

Roughly 320 MB, one time. Re-check with `ollama list`, then index again.

If you deliberately use a different model, pass it per command (`lumen index -m <model>`, `lumen search -m <model>`) and be aware that Lumen maintains a **separate index per model** — the same project will be embedded twice.

---

7. Newly created or edited files are missing from search results

Symptom

You add `src/new_feature.py`, search for something obviously in it, and get nothing — while older files in the same repository match fine.

Diagnosis

Lumen walks the file tree at the moment indexing **starts**. Files created after that walk began are not part of that pass, no matter how long the pass runs. An index that finished "just now" can therefore be missing a file you created two minutes ago.

This is also why an agent that writes files and immediately searches for them comes up empty.

Fix

Re-run the indexer. It is incremental — unchanged files are not re-embedded, so this is cheap:

```
lumen index /path/to/repo
```

Force a complete rebuild only when you suspect the index itself is damaged:

```
lumen index /path/to/repo -f
# or, in one step, at search time:
lumen search "..." -p /path/to/repo -f
```

Habit that avoids the problem: index *after* a batch of file creation, not before.

---

8. `jq` missing

Symptom

```
jq: command not found
```

or the wizard stops in Step 1 with `Could not install jq. Please install it manually.`

Diagnosis

`jq` is what merges MCP servers into every agent config; the wizard refuses to continue without it. `ensure_jq` tries to install it automatically:

Platform	Command it runs
Linux with `apt-get`	`sudo apt-get update -qq && sudo apt-get install -y jq`
Linux with `yum`	`sudo yum install -y jq`
macOS with `brew`	`brew install jq`
macOS without `brew`	Installs Homebrew, then `brew install jq`
Anything else	Prints an error and exits `1`

So this only surfaces when you are on a distribution with neither `apt-get` nor `yum` (pacman, zypper, apk, Nix, …), or when `sudo` is unavailable.

Fix

Install `jq` with your own package manager and re-run the wizard:

```
sudo pacman -S jq        # Arch
sudo zypper install jq   # openSUSE
sudo apk add jq          # Alpine
```

Then confirm: `jq --version`. The wizard is safe to re-run — see FAQ: is it safe to re-run.

---

9. Duplicate PATH entries after several runs

Symptom

Suspicion that running the wizard N times prepends `~/.local/bin` to `PATH` N times.

Diagnosis

It cannot. Two independent mechanisms prevent it:

1. **The blocks are replaced, not appended.** `strip_managed_block` removes the previous marker-delimited block before the new one is written, so `~/.bashrc` holds exactly one Lumen block and `~/.bash_profile` exactly one user-bin block, no matter how many runs (tests **C3**, **C4**).

2. **The line itself is guarded.** Both blocks contain:

```
case ":$PATH:" in *":$HOME/.local/bin:"*) ;; *) export
        PATH="$HOME/.local/bin:$PATH";; esac
```

The `case` matches only when the entry is already surrounded by colons in `PATH`, so sourcing the file repeatedly — `.bash_profile` sourcing `.bashrc`, a nested shell, `exec bash -l` — is a no-op after the first time (test **C8**: exactly one occurrence).

Note the guard is written to disk *literally*, with `$PATH` and `$HOME` unexpanded (test **C7**). If your `~/.bashrc` contains a real path like `/home/someone/.local/bin` instead of `$HOME`, that block was not written by this wizard.

Check it yourself

```
tr ':' '\n' <<<"$PATH" | sort | uniq -d          # prints nothing
        when there are no duplicates
tr ':' '\n' <<<"$PATH" | grep -cx "$HOME/.local/bin"   # -> 1
```

If a duplicate does show up, it came from elsewhere. Find the culprit:

```
grep -n "local/bin" ~/.bashrc ~/.bash_profile ~/.profile
        ~/.bash_login 2>/dev/null
```

Lines outside the `# >>> ... >>>` / `# <<< ... <<<` markers are not managed by the wizard and will not be removed by it.

---

10. `lumen: command not found` right after the wizard finishes

Symptom

The wizard's summary shows `launcher (/home/<you>/.local/bin/lumen)` but the very next command in the same terminal says `lumen: command not found`.

Diagnosis

The wizard modified `~/.bashrc` and `~/.bash_profile`; your current shell read those files before they changed. `PATH` in an already-running shell is not updated by editing rc files.

```
ls -l ~/.local/bin/lumen        # exists and is mode 755?
~/.local/bin/lumen version      # works when called by absolute
        path?
echo "$PATH" | tr ':' '\n' | grep -x "$HOME/.local/bin"
```

If the absolute path works and the `grep` prints nothing, this is the cause.

Fix

```
exec bash -l        # or: . ~/.bashrc, or open a new terminal
command -v lumen
```

Full detail, including zsh/fish and the non-interactive cases: FAQ: why isn't lumen found and FAQ: cron / ssh / systemd.

---

11. Collecting evidence for a bug report

Do not describe the failure — reproduce it with the test suite and attach the evidence directory. Every assertion records the expected value, the actual value and a timestamp.

```
cd /path/to/this/repo
./scripts/test-setup-agents-wizard.sh
```

It prints the evidence location and a summary:

```
================ EVIDENCE ================
  results.tsv : .test-evidence/<UTC timestamp>/results.tsv
  run.log     : .test-evidence/<UTC timestamp>/run.log
  summary.json: .test-evidence/<UTC timestamp>/summary.json
  total=... passed=... failed=... skipped=...
==========================================
```

Attach the whole `.test-evidence/<timestamp>/` directory:

File	Contents
`results.tsv`	One row per assertion: id, group, name, PASS/FAIL/SKIP, expected, actual, timestamp
`run.log`	Full console transcript, including group headers and failure detail
`summary.json`	Host (`uname`, bash version), wizard path, totals, exit code

Useful facts about the suite before you run it:

- There are **ten** groups, **A**–**J**. The file declares **H** before **G**, so the console order is A, B, C, D, E, F, H, G, I, J.

- Groups **A** and **I** read source text only — no sandbox, no execution. Groups **B**, **C**, **D**, **G** and **H** run against a throwaway `$HOME` (with `WIZARD_STATE_DIR` pointed inside it), and groups **E** and **J** against throwaway git repositories. None of them touches your real environment.

- Group **I** covers the three operational scripts (`ollama-vulkan-remediation.sh`, `lumen-reindex.sh`, `lumen-index-doctor.sh`) and group **J** covers `audit-hardcoded-paths.sh` — see OPERATIONAL-SCRIPTS.md.

- **Do not quote a fixed total in a bug report** — quote `summary.json`. The count changes as tests are added, and `--no-live` records fewer. For reference, a full live run currently produces 138 records (A 54, B 8, C 10, D 5, E 3, F 11, G 18, H 7, I 14, J 8) and `--no-live` produces 134. Note `A`, `F` and `I` emit more records than they have ids, because `A12`, `F2` and `I1`–`I3` are loops.

- Group **F** is the live one: it checks `PATH` resolution in login/interactive shells, backend reachability, the embedding model, and performs a true end-to-end index + search on a two-file fixture repository in a temp directory (which it purges afterwards). Skip it with:

  ```
  ./scripts/test-setup-agents-wizard.sh --no-live
  ```

  `--no-live` records `F1`–`F7` as skips; `F8` is simply not recorded in that mode, so do not read its absence from `results.tsv` as a failure.

- Exit status is non-zero if any assertion failed, so it is CI-usable as-is.

Add these to the report when the failure is environment-specific:

```
lumen version
uname -s -m
bash --version | head -1
jq --version
systemctl is-active ollama
ollama list
du -sh ~/.local/share/lumen
claude mcp get lumen
jq -c '.mcpServers' ~/.kimi-code/mcp.json ~/.qwen/settings.json
jq -c '.mcp'        ~/.config/opencode/opencode.json
sed -n '/>>> lumen semantic search/,/<<< lumen semantic search/p'
        ~/.bashrc
./scripts/rollback-agents-wizard.sh --list
```

Redact paths if the repository name is sensitive — `results.tsv` and `run.log` contain the project root and your `$HOME`.

---

12. Restoring from a backup session

Symptom

A wizard run changed something you wanted left alone — a shell file, an agent config, the `lumen` wrapper — and you want it back exactly as it was.

Diagnosis

You do not need to reconstruct anything by hand. The wizard opens a **backup session** before Step 1 and records every mutation in it. Find the sessions:

```
./scripts/rollback-agents-wizard.sh --list

========================================
 Backup sessions
========================================
  20260826T210411Z      10 changes   components: claude kimi lumen
npm opencode qwen shell
  20260827T090300Z       4 changes   components: claude shell
```

Each session is **one wizard run**, holding a `manifest.tsv` and byte-exact copies of the originals under `files/`. Read the manifest directly if you want the detail:

```
column -t -s $'\t' ~/.local/share/setup-agents-wizard/backups/
        latest/manifest.tsv
```

Fix

Preview first — `--dry-run` prints the plan and changes nothing:

```
./scripts/rollback-agents-wizard.sh --dry-run
```

Then apply, either wholesale or scoped to one component:

```
./scripts/rollback-agents-wizard.sh --yes
        # the whole newest run
./scripts/rollback-agents-wizard.sh -c shell --yes
        # only ~/.bashrc + ~/.bash_profile
./scripts/rollback-agents-wizard.sh -c opencode --yes
        # only ~/.config/opencode/opencode.json
./scripts/rollback-agents-wizard.sh --session 20260826T210411Z --
        yes   # an older run
./scripts/rollback-agents-wizard.sh --run-actions --yes
        # also run the npm / SpecKit / claude mcp undo commands
exec bash -l
        # pick up the restored PATH
```

Things that surprise people:

- **`ACTION` rows are printed, not executed.** An `npm install -g` or the `claude mcp` registration appears as `manual [npm] npm uninstall -g glyphdown`. Pass `--run-actions` to have the tool run them.
- **The rollback is reversible.** Current state is copied into `<session>/pre-rollback-<UTC>/` before anything is touched, so a rollback you regret can be undone too.
- **Restart the agents.** MCP servers are launched at session start, so a restored config only takes effect on the next session.

Full reference, including how to reverse a rollback file by file: SAFETY-AND-ROLLBACK.md.

---

13. Rollback says the backup is missing

Symptom

```
Backup missing for /home/<you>/.bashrc (/home/<you>/.local/
share/setup-agents-wizard/backups/20260826T210411Z/files/
9f2c1ab30e7d4415__.bashrc)
```

or, before it even gets that far:

```
No backup sessions found at /home/<you>/.local/share/setup-
agents-wizard/backups
 The wizard creates one every time it runs.
```

The first case exits **1** and reports the count under `failures:`; other entries in the same run still apply.

Diagnosis

The manifest records **absolute** paths, so the stored copy has to be exactly where the row says it is. Three things break that:

1. **`WIZARD_STATE_DIR` differs between the two scripts.** If the wizard ran with a custom value, the rollback tool needs the same one — it defaults to `$HOME/.local/share/setup-agents-wizard`.
2. **The state directory was moved, renamed or partially deleted.** Copying `backups/` elsewhere invalidates every `backup` column.
3. **The wizard could not make the copy in the first place.** It prints `Could not back up <path>` at the time and writes no row — so if a row exists, a copy was made.

Check what is actually there:

```
echo "${WIZARD_STATE_DIR:-$HOME/.local/share/setup-agents-wizard}"
ls -d ~/.local/share/setup-agents-wizard/backups/*/
SESSION=~/.local/share/setup-agents-wizard/backups/latest
awk -F'\t' 'NR>1 && $4!="-" {print $4}' "$SESSION/manifest.tsv" |
        while read -r b; do
  [ -f "$b" ] || echo "MISSING: $b"
done
```

Fix

Point both scripts at the same directory:

```
WIZARD_STATE_DIR=/path/you/used ./scripts/rollback-agents-
        wizard.sh --list
```

If the copies really are gone, rollback cannot restore those entries. Fall back to:

- **The `.bak.<ts>` siblings**, which `backup_file` writes next to each file it touches and which rollback never deletes:

  ```
  ls -1t ~/.bashrc.bak.* | head -1        # newest first
  cp -p ~/.bashrc.bak.20260826210427 ~/.bashrc
  ```

  Note `~/.local/share/bash-completion/completions/lumen` has no sibling — it is registered with `snapshot_before`, not `backup_file`.

- **An earlier session**, if one exists — `--list`, then `--session <id>`.

- **The manual uninstall procedure**, component by component: SAFETY-AND-ROLLBACK.md → Manual uninstall.

To avoid the problem next time, keep the state directory where the wizard put it, and note that a `CREATED` row needs no backup file at all — those entries roll back cleanly even when `files/` is empty.

---

14. The index is silently corrupt (stale duplicate vectors)

Symptom

There isn't one. That is the point.

Searches return results. They are plausible. They are also wrong — or a file you *know* is indexed never comes back for a query it should obviously match. No error is printed anywhere, `lumen index` exits `0`, `/api/embed` answers `200`, and every health check in this document passes.

Diagnosis

If `ollama` was ever serving the embedding model with `library=Vulkan` on an Intel iGPU, a large embedding **batch** can trip an i915 fence timeout. The kernel abandons the dispatch and ollama reads the result buffer anyway. Four things can come back:

#	What comes back	HTTP	Caught by a per-vector check?
1	`{"error":"…unsupported value: NaN"}`	500	Yes — see #5b
2	An all-zero vector	**200**	Yes — a zero-norm test sees it
3	The runner wedges: every later request returns `NaN`	500	Yes, usually misattributed
4	**A repeated STALE vector** — well-formed, 768 dims, L2 norm 1.000000	**200**	**No.**

Mode 4 wrote **758 identical vectors covering 695 distinct texts across 55 files** into this project's index. A full forensic audit ran NaN, Inf, all-zero and L2-norm checks, found nothing, and declared the index `TRUSTWORTHY`. Its measurements were correct; its conclusion was not — all four tests are *per-vector*, and a stale-but-well-formed vector passes all four by construction.

> **A vector can be perfectly well-formed and still be the wrong vector.** The only thing that exposes mode 4 is **aggregate distinctness**: *N* distinct texts must yield *N* distinct vectors.

Note also that the trigger is the **batch total**, not the chunk size. Lumen sends 32 chunks per `/api/embed` request; the corrupting requests here carried 16,698–31,364 characters, while the largest single chunk involved was only 2,832. Capping chunk size does not protect you.

Check it — read-only, safe while indexing:

```
./scripts/lumen-index-doctor.sh "$PWD"
#   exit 0 = healthy · 1 = corruption found · 2 = could not
      inspect
```

A corrupt index looks like this:

```
vectors: 35717 total, 34959 distinct
   1 duplicate-vector group(s); 758 vectors (2.12%) are not unique
   largest identical group: 758 copies of ONE vector
per-vector: 0 NaN/Inf, 0 all-zero, 0 off-norm

   CORRUPTION DETECTED
```

Note the last line of `per-vector:` — **all zeros**. That is exactly what the earlier audit saw.

Two things about that exit code, before you script around it: a duplicate group only sets exit `1` when its **largest** member count is ≥ 10, so a small group prints a line and still exits `0`; and an internal error in the decoder (if a future Lumen changes the 768-dimension layout or the `vec_chunks_vector_chunks00` shadow-table name) also exits `1`. Read the output, not just the status.

Fix

Order matters. Fixing the backend repairs nothing already written, and rebuilding onto a broken backend just writes fresh corruption.

```
# 1. Is the backend still the corrupting one?
./scripts/ollama-vulkan-remediation.sh --check
#    expect: "library=cpu" and "batch probe: OK 32/32 distinct"

# 2. If it says library=Vulkan — fix it. sudo, one service
      restart.
./scripts/ollama-vulkan-remediation.sh --apply

# 3. Rebuild. --force is MANDATORY, not a preference.
./scripts/lumen-reindex.sh "$PWD" --force

# 4. Confirm
./scripts/lumen-index-doctor.sh "$PWD"     # expect exit 0
```

Why `--force` is mandatory: a corrupted file still carries a valid content hash in the index, so Lumen treats it as already done. An incremental run skips it — **forever**. Only `lumen index -f` re-embeds it.

`scripts/lumen-reindex.sh` refuses to start at all if the backend still reports `library=Vulkan` (exit 3), and refuses if its own 32-text batch probe fails (exit 4). Both refusals are the script doing its job; do not reach for `--allow-gpu` to get past them unless you have decided you want the risk.

What does not help

- `OLLAMA_VULKAN=false` and `OLLAMA_LLM_LIBRARY=cpu` — both tested, both **no-ops** on this build. `GGML_VK_VISIBLE_DEVICES=-1` is the variable that works.
- Upgrading ollama — the defect is unfixed upstream.
- `ollama stop <model>` — clears a wedged runner (mode 3), does nothing for modes 2 and 4.
- Capping `LUMEN_MAX_CHUNK_TOKENS` — the trigger is the batch total, not the chunk.
- Any per-vector integrity check you can think of.

Reference

- OPERATIONAL-SCRIPTS.md — every flag and exit code of the three scripts.
- OLLAMA-REMEDIATION.md — the operator runbook. **Status on this host: resolved** — `library=cpu`, 0 i915 faults since the restart, 32 distinct vectors 5/5.
- INDEX-CORRUPTION-RECONCILIATION.md — **the settled verdict**, with the per-file breakdown of all 758 vectors.
- LUMEN-INDEX-INTEGRITY.md — the earlier audit that concluded `TRUSTWORTHY`. Kept unedited as a record of how a rigorous audit reached a wrong conclusion. **Do not use it to decide whether an index is safe.**